

Tema 10 - Introducción a la biblioteca estándar del lenguaje

Contenido Teórico

Objetivos

- Entender qué aporta la biblioteca estándar y cuándo usarla antes de instalar paquetes externos.
- Localizar documentación oficial y leer ejemplos de uso de un módulo.
- Usar módulos útiles para automatización y soporte: `pathlib`, `shutil`, `subprocess`, `logging`, `argparse`, `datetime`, `csv`, `json`, `sqlite3`, `urllib` y seguridad básica.
- Aplicar criterios de selección: estabilidad, portabilidad, mantenimiento y seguridad.
- Integrar varios módulos estándar en scripts pequeños orientados a entornos de TI.

Qué es la biblioteca estándar

La biblioteca estándar es el conjunto de módulos que se distribuyen con Python.

Aporta herramientas para tareas comunes: rutas, fechas, formatos de datos, logs, procesos, red, seguridad, compresión, bases de datos ligeras y utilidades de desarrollo.

No es necesario instalarlos con `pip`. La biblioteca estándar viene incluida con Python y sus módulos están disponibles al usar el intérprete.

No todo lo que se usa en Python forma parte de la biblioteca estándar. Por citar algunos paquetes externos muy usados: `numpy` para matrices y cálculo matemático, `pandas` para manipulación de tablas, `matplotlib` para gráficos, `scikit-learn` para modelos de Machine Learning, `requests` para peticiones HTTP y `graphviz` para diagramas.

Los paquetes externos se gestionan con `pip` y deben quedar documentados en el proyecto.

```
# Biblioteca estándar
import csv
import json
import sqlite3

# Paquetes externos
# import pandas
# import requests

# Documentar externos:
# pip freeze > requirements.txt
```

Documentación oficial y lectura de módulos

La referencia principal es la documentación oficial de Python: <https://docs.python.org/3.13/library> para inglés y <https://docs.python.org/es/3.13/library/> para español.

También podemos consultarlo directamente desde la terminal de trabajo. Por ejemplo: `python -m pydoc json`.

Para cada módulo conviene revisar la descripción, las funciones principales, los ejemplos, las excepciones y las notas de seguridad.

Antes de escribir código desde cero o instalar un paquete externo, conviene hacerse cuatro preguntas: si existe un módulo estándar que cubra la tarea, si es suficientemente legible para el equipo, si funciona de forma portable en el entorno y si tiene limitaciones de seguridad o rendimiento.

Esta disciplina reduce dependencias, facilita soporte y portabilidad, y hace que los scripts sean más fáciles de ejecutar en sistemas controlados.

En este tema veremos ejemplos pequeños y ejecutables, pero en proyectos reales la documentación debe ser la fuente de validación.

```
import json
import shutil
from pathlib import Path

print(json.__doc__.splitlines()[0])
print(json.dumps(
    {"servicio": "ssh", "puerto": 22}))

origen = Path("data/config.json")
destino = Path("backup/config.json")

# Mejor que copiar bytes manualmente
destino.parent.mkdir(exist_ok=True)
shutil.copy2(origen, destino)

# En terminal:
# python -m pydoc json
```

Familias útiles en administración TI

Podemos agrupar la biblioteca estándar por tipo de aplicación y uso en entornos TI.

- Sistema y entorno: sys, os, platform.
- Ficheros y rutas: pathlib, shutil, glob, tempfile.
- Procesos: subprocess.
- Datos: csv, json, configparser, tomlib.
- Operación: logging, datetime, zoneinfo.
- Seguridad básica: hashlib, hmac, secrets.
- Persistencia ligera: sqlite3.
- Acceso HTTP básico: urllib.

```
modulos = {
    "entorno": ["sys", "os", "platform"],
    "datos": ["csv", "json", "sqlite3"],
    "operacion": ["logging", "argparse"],
}

for area, nombres in modulos.items():
    print(area, "->", ", ".join(nombres))
```

Sistema y entorno: os, sys y platform

Los scripts de administración suelen necesitar información sobre el entorno de ejecución: versión de Python, sistema operativo, usuario, arquitectura o directorio actual.

`os`, `sys` y `platform` permiten obtener esos datos sin instalar herramientas externas.

Los módulos `sys` y `platform` ayudan a inspeccionar el entorno de ejecución.

El módulo `os` permite leer variables de entorno, como credenciales o rutas, e interactuar con el sistema operativo a nivel de proceso.

Son útiles para scripts de soporte que necesitan registrar versión de Python, sistema operativo o arquitectura.

No sustituyen a una herramienta de inventario, pero son suficientes para diagnósticos rápidos y para registrar contexto en logs.

```
import os
import sys
import platform

# sys: Información del intérprete Python
print("Python:", sys.version.split()[0])

# os: Información del entorno y procesos
print("Usuario:", os.environ.get("USER"))
print("PID actual:", os.getpid())

# platform: Detalles del hardware y OS
print("Sistema:", platform.system())
print("Release:", platform.release())
print("Máquina:", platform.machine())
```

Rutas y ficheros: pathlib, shutil y glob

`pathlib` permite construir rutas legibles y portables. Ya se estudió en el tema de entrada-salida.

`shutil` ofrece operaciones de alto nivel, como copiar ficheros o directorios completos.

`glob` ayuda a localizar ficheros por patrón.

Estos módulos son muy útiles para scripts de soporte: recopilar logs, preparar carpetas de salida, copiar configuraciones o buscar archivos antiguos.

El objetivo es evitar concatenar rutas como cadenas y reducir errores dependientes del sistema operativo.

```
from pathlib import Path
import shutil
import glob

# Rutas relativas asumiendo ejecución desde Curso_Python
base_dir = Path("Temal0")
data_dir = base_dir / "data"
backup_dir = base_dir / "backup"

# Creación de un fichero en /data
data_dir.mkdir(parents=True, exist_ok=True)
(data_dir / "config1.json").write_text('{"estado": "activo"}')
(data_dir / "config2.json").write_text('{"estado": "activo"}')
```

```
# Copia backup de fichero con shutil
shutil.copytree(data_dir, backup_dir, dirs_exist_ok=True)

# Búsqueda de ficheros con glob
archivos = glob.glob(str(backup_dir / "*.json"))
print(f"Ficheros respaldados: {archivos}")
```

Temporales y contextos: tempfile

`tempfile` permite crear ficheros y directorios temporales de forma segura.

Es útil para pruebas, procesamiento intermedio o scripts que deben limpiar automáticamente recursos.

Cuando trabajamos con ficheros temporales, no conviene improvisar nombres manuales en directorios compartidos.

```
import tempfile
from pathlib import Path

# Aseguramos la existencia de la ruta base del laboratorio
ruta_base = Path("Temal0/data")
ruta_base.mkdir(parents=True, exist_ok=True)

# Forzamos la creación del temporal dentro de nuestro subdirectorio
with tempfile.TemporaryDirectory(dir=ruta_base) as tmp_dir:
    directorio = Path(tmp_dir)
    print(f"Subdirectorio temporal creado en: {directorio}")
    archivo = directorio / "dump_db.sql"
    archivo.write_text("SELECT * FROM usuarios;", encoding="utf-8")
    print(f"Dentro del contexto. ¿Existe el archivo?: {archivo.exists()}")

# Al salir del bloque, se ejecuta la limpieza automática
print(f"Fuera del contexto. ¿Sigue existiendo?: {directorio.exists()}")
```

Procesos externos con subprocess

`subprocess` permite ejecutar comandos del sistema desde Python y capturar su salida. Es útil para integrar herramientas ya existentes.

Debe usarse con cuidado: conviene pasar argumentos como lista, validar entradas y revisar el código de retorno.

Evitar `shell=True` salvo necesidad justificada reduce riesgos de seguridad. El valor predeterminado es `False`.

Puede servir para integrar herramientas existentes, siempre controlando errores y salida. Conviene capturar `stdout` y `stderr`.

```
import subprocess

# Tarea de infraestructura: comprobar espacio en disco
comando = ["df", "-h", "/"]
```

```

resultado = subprocess.run(
    comando,
    capture_output=True,
    text=True,          # Salida UTF-8
    check=False         # Si falla Python no lanza excepción
)

# Controlamos el flujo basándonos en el código de retorno
if resultado.returncode == 0:
    print("Comando exitoso. Salida estándar (stdout):")
    print(resultado.stdout.strip())
else:
    print("Error en la ejecución. Detalle (stderr):")
    print(resultado.stderr.strip())

```

Formatos de datos: csv, json, configparser y tomlib

La biblioteca estándar incluye módulos para los formatos habituales.

csv es útil para exportaciones tabulares. json es habitual en APIs y configuraciones. configparser permite leer INI. tomlib lee TOML en Python moderno.

En scripts, estos módulos permiten intercambiar datos con herramientas existentes sin depender de paquetes externos.

El formato INI es plano y lee absolutamente todo como texto. TOML es un estándar más moderno para ficheros de configuración: reconoce números, listas, booleanos y subsecciones anidadas.

```

import configparser
import tomlib
from pathlib import Path

carpeta = Path("Temal0/data")
carpeta.mkdir(exist_ok=True)

# Lectura de un formato clásico: .ini
config_ini = configparser.ConfigParser()
config_ini.read(carpeta / "database.ini")

puerto_db = config_ini["mysqld"]["port"]
print(f"Puerto extraído de INI: {puerto_db}")

# Lectura de un formato moderno: .toml
ruta_toml = carpeta / "app_config.toml"
with open(ruta_toml, "rb") as filetoml:
    config_toml = tomlib.load(filetoml)

modo_debug = config_toml["server"]["debug_mode"]
print(f"Modo debug de TOML: {modo_debug}")

```

Logs con logging

print() es suficiente para aprender, pero no para operar scripts reales.

logging permite registrar fecha, nivel, módulo y mensaje.

Los niveles ayudan a separar información normal, avisos y errores: DEBUG, INFO, WARNING, ERROR y CRITICAL.

Una buena práctica es registrar el contexto técnico en un log sin llenar la salida del usuario con trazas innecesarias.

```
from pathlib import Path
import logging

carpeta = Path("Tema10/data")
carpeta.mkdir(exist_ok=True)

logging.basicConfig(
    filename="Tema10/data/soporte.log",
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
)

logging.info("Inicio de comprobación de infraestructura.")
logging.warning("Latencia alta detectada en el servicio API.")

try:
    raise ConnectionRefusedError("Conexión rechazada por el puerto 5432")
except ConnectionRefusedError as e:
    logging.error(f"Fallo en nodo srv-db-01: {e}")
    print("Error: La base de datos no está disponible en este momento.")
```

Fechas, horas y zonas: datetime y zoneinfo

datetime permite trabajar con fechas, horas e intervalos.

zoneinfo añade soporte de zonas horarias usando la base de datos disponible en el sistema.

calendar permite consultar calendarios y propiedades de años o meses.

En administración de sistemas es habitual calcular ventanas de mantenimiento, antigüedad de ficheros, fechas de reporte, marcas de tiempo o nombres de informes.

Conviene usar formatos claros como ISO 8601 cuando se van a intercambiar datos entre sistemas.

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo
import calendar

# Obtener año y mes actual en UTC
ahora_utc = datetime.now(timezone.utc)
anio, mes = ahora_utc.year, ahora_utc.month

# Calcular el último día del mes con calendar
_, ultimo_dia = calendar.monthrange(anio, mes)

# Definir ventana de ejecución el último día del mes a las 22:00
zona_local = ZoneInfo("Europe/Madrid")
ejecucion_local = datetime(anio, mes, ultimo_dia, 22, 0, tzinfo=zona_local)

print(f"Log del sistema (UTC): {ahora_utc.isoformat(timespec='seconds')}")
print(f"Planificación (Local): {ejecucion_local.isoformat(timespec='seconds')}")
```

Seguridad básica: hashlib, secrets y random

`hashlib` calcula huellas criptográficas de datos.

`secrets` genera valores aleatorios adecuados para tokens o credenciales temporales. Utiliza el generador criptográfico del kernel.

`random` es un generador pseudoaleatorio útil para simulaciones, pruebas y selección no crítica. No debe usarse para generar contraseñas, tokens o valores de seguridad.

La biblioteca estándar no resuelve toda la seguridad, pero sí cubre tareas básicas con garantías razonables.

```
import hashlib
import secrets
import random

# secrets: generación criptográfica para credenciales
pwd_temporal = secrets.token_urlsafe(12)
print(f"Token temporal seguro: {pwd_temporal}")

# hashlib: cálculo de huella unidireccional
huella_sha256 = hashlib.sha256(pwd_temporal.encode("utf-8")).hexdigest()
print(f"Hash (SHA-256): {huella_sha256}")

# random: uso estadístico NO crítico
servidor = random.choice(["srv-web-01", "srv-web-02", "srv-web-3"])
print(f"Desplegado aleatoriamente en: {servidor}")
```

Persistencia ligera con sqlite3

`sqlite3` permite usar una base de datos local en un único fichero, sin instalar un servidor.

Es útil para scripts que necesitan guardar resultados, auditorías o pequeños inventarios.

No sustituye a un motor corporativo, pero evita usar CSV cuando ya necesitamos consultas estructuradas.

Como siempre, los datos externos deben pasarse con parámetros, no concatenando SQL.

```
import sqlite3
from pathlib import Path

carpeta = Path("Tema10/data")
carpeta.mkdir(exist_ok=True)
bd = sqlite3.connect("Tema10/data/inventario.db")

bd.execute("CREATE TABLE IF NOT EXISTS nodos (host TEXT, ip TEXT, rol TEXT)")

datos = [
    ("web-01", "10.0.1.10", "frontend"),
    ("db-01", "10.0.1.20", "backend"),
    ("cache", "10.0.1.30", "redis")
]
bd.executemany("INSERT INTO nodos VALUES (?, ?, ?)", datos)
bd.commit()

for fila in bd.execute("SELECT host, ip FROM nodos WHERE rol = ?", ("backend",)):
    print(f"Servidor crítico: {fila[0]} localizado en {fila[1]}")

bd.close()
```

HTTP básico con urllib

urllib permite realizar peticiones HTTP sin instalar paquetes externos.

Para usos sencillos puede ser suficiente, especialmente si solo se necesita leer una URL interna o consultar un endpoint básico.

En proyectos más complejos se suelen usar paquetes externos como requests.

En cualquier caso hay que controlar tiempos de espera, errores y formato de respuesta.

```
from urllib.request import urlopen
from urllib.error import URLError
import json

# API pública para obtener la IP
url = "https://api.ipify.org?format=json"

try:
    with urlopen(url, timeout=5) as respuesta:
        datos = json.load(respuesta)
        print(f"Conexión exitosa. Tu IP es: {datos['ip']}")
except URLError as error:
    print(f"Error al conectar con la API: {error.reason}")
```

Buenas prácticas y errores frecuentes

Usar módulos estándar no significa escribir código improvisado. Conviene mantener imports ordenados, funciones pequeñas, control de errores y documentación mínima.

Buenas prácticas

- Preferir módulos estándar cuando resuelven bien el problema.
- Leer la documentación y las notas de seguridad.
- Mantener imports ordenados.
- Controlar excepciones.
- Registrar información operativa con logging.

Errores frecuentes

- Confundir biblioteca estándar con paquetes externos.
- Usar nombres de fichero como `json.py`, `random.py` o `logging.py`.
- Instalar dependencias externas innecesarias.
- No controlar errores de red, procesos o ficheros.
- Usar APIs antiguas por costumbre cuando existen alternativas más claras, como `Path` o `logging`.

```
# Error frecuente: nombre de fichero peligroso
# json.py, random.py, logging.py

# Puede ocultar el módulo estándar real
import json

# Mejor: nombres descriptivos propios
# utilidades_json.py
# generar_reporte.py
```


Resumen del Tema 10

La biblioteca estándar es una colección de módulos incluidos con Python para resolver tareas comunes sin instalar dependencias externas.

En administración TI destacan módulos para sistema, ficheros, procesos, logs, fechas, formatos de datos, colecciones especializadas, seguridad, bases de datos ligeras y acceso HTTP básico.

La idea clave no es memorizar módulos, sino saber buscar, seleccionar y combinar herramientas estándar con criterio profesional.

Flujo de trabajo en el laboratorio

Desde el repositorio Git podrás cargar el directorio del Tema10.

Los laboratorios usarán módulos estándar sin instalar paquetes externos. La intención es consolidar la consulta de documentación, la lectura de errores y la composición de módulos en scripts pequeños.

La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python
source .venv/bin/activate
python --version

# Actualizar del repositorio Git
git pull origin main

# Para ejecutar VS Code
code . # usar carpeta Tema10

# Para ejecutar en terminal
python
```